

**Project report
on
MockU – A one stop Mock Tests Platform**

**Project report submitted in partial fulfilment of the requirement for the award
of the Degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted By

KT Karthik [R210058]

M Ranjith Sai [R210436]

K Dharshan Sai [R210431]

Under the Esteemed Guidance of

**Ms.E.Susmitha, M.Tech, (Ph.D.)
Assistant Professor**



RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP IIIT)

R.K Valley, Vempalli, Kadapa(Dist) – 516 330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-2026

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES (AP IIIT)

R.K Valley, Vempalli(M), Kadapa(Dist) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-2026



CERTIFICATE

This is certify that project report entitled **“MOCKU -A One stop Mock Test Platform”** being submitted by KT Karthik, M Ranjith,K Dharshan under my guidance & Supervision and is submitted to **DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING** in partial fulfilment of requirement of the award of the Bachelor of Technology in CSE during the academic year 2025-2026 and it has found worthy of acceptance according to the requirements of the University.

Internal Guide

**Ms, E. Susmitha,
M.Tech., (Ph.D.),
Assistant Professor
Department of CSE
RGUKT - RK Valley**

Head of the Department

**Dr.Ch. Ratna Kumari,
M.Tech., Ph.D.,
Assistant Professor
Department of CSE
RGUKT - RK Valley**

Submitted on:

Signature of External Examiner

ACKNOWLEDGEMENT

I wish to express our sincere thanks to various personalities who were responsible for successful completion of the major project.

I am grateful to **Dr. Ch. Ratna Kumari, M.tech., Ph.D.**, Head of the Department , for her motivation and encouragement in completing the project in specified time.

I express my default gratitude to **MS. E. Susmitha, M.Tech, (Ph.d.)** internal guide for valuable guidance and encouragement which enabled me to the successful completion of project in time.

I express my sincere thanks to all other faculty members of the CSE Department for extending their helping hands and valuable suggestion when in need.

Finally, my heartfelt thanks to my parents for giving me all I ever needed to be a successful student and individual. Because of their hard work and dedication, I have had oppurtunities beyond my wildest dreams.

WITH SINCERE REGARDS

KT Karthik	[R210058]
K Dharshan Sai	[R210431]
M Ranjith Sai	[R210436]

DECLARATION

Hereby declare that the major project work entitled **“MockU-One stop Platform”** Submitted to **DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING** is a genuine work carried out by us , for the fulfillment of the Bachelors of Technology in the Department of CSE during the academic year 2025-2026 under the supervision of the project guide , Mrs. E. SUSMITHA Assistant Professor , Department of CSE in RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES (AP-IIIT) ,RK-Valley

WITH SINCERE REGARDS

KT Karthik [R210058]

K Dharshan Sai [R210431]

M Ranjith Sai [R210436]

ABSTRACT

Competitive examinations such as GATE, UPSC, CAT, GMAT, JEE, NEET, and EAMCET demand strategic preparation, continuous evaluation, and focused improvement. Although several online platforms provide mock tests, most of them limit their functionality to score display and ranking, without offering meaningful guidance on performance improvement. This lack of structured feedback often leaves aspirants unaware of their weak areas, preparation gaps, and effective exam strategies. The proposed project, MockU, aims to provide a comprehensive and intelligent exam preparation environment by integrating mock testing, performance analysis, and guided feedback into a single platform.

The system enables users to attempt exam-specific mock tests designed according to respective exam patterns, marking schemes, and sectional structures. After test completion, the platform evaluates user performance using AI-assisted analysis, predefined analytical rules, and assessment logic. MockU identifies strong and weak sections, topic-wise performance, accuracy levels, time management efficiency, and common mistake patterns. Based on this analysis, it provides personalized guidance such as recommended focus areas, do's and don'ts for each examination, preparation strategies, and revision suggestions. Additional features include progress tracking dashboards, attempt history, syllabus coverage monitoring, and mistake review modules to support continuous improvement. The proposed platform enhances exam preparation by shifting focus from mere score evaluation to structured performance guidance and strategic learning, making it suitable for competitive exam aspirants, coaching institutes, and self-learning platforms.

MockU provides an intelligent and structured approach to competitive exam preparation by combining mock testing with performance analysis and guided feedback. The system helps students understand their preparation gaps, improve weak areas, and track progress effectively. By focusing on performance-driven learning rather than only score display, MockU enhances preparation efficiency and supports aspirants in achieving better academic outcomes.

Technologies Used:

React Native Express Node MongoDB RestAPI

CONTENTS

TITLE	I
CERTIFICATE	II
ACKNOWLEDGEMENT	III
DECLARATION	IV
ABSTRACT	V

CH.NO .	TITTLE NAME	PAGE NO.
1.	INTRODUCTION	1-2
2.	SYSTEM ARCHITECTURE	3-6
3.	TECHNOLOGY STACK	7
4.	FEATURES AND FUNCTIONALITIES	8-12
5.	DATABASE DESIGN	13-15
6.	IMPLEMENTATION DETAILS	16-17
7.	USER INTERFACE AND DESIGN	18-19
8.	SECURITY FEATURES	20
9.	FUTURE ENHANCEMENTS	21
10.	CONCLUSION	22
11.	REFERENCES	23

INTRODUCTION

1. Project Overview

1.1 Project Title

MockU – Online Mock Test Platform for Competitive Exam Preparation

1.2 Problem Statement

In today's competitive environment, students preparing for exams like SSC, Banking, UPSC, and Engineering entrance tests require consistent practice, performance tracking, and real-time feedback. Traditional preparation methods often lack structured mock testing, detailed analytics, and centralized platforms for multiple exams.

- 1) Real-time bidirectional communication
- 2) Friend request and management system
- 3) Multiple chat room types (Private, Group, Global)
- 4) Multimedia content sharing (images, documents, camera photos)
- 5) Rich emoji support
- 7) Online/offline status tracking
- 8) Responsive, user-friendly interface

1.3 Objectives

Primary Objectives:

- 1) Develop a scalable mock test platform using modern web/mobile technologies
- 2) Provide realistic test-taking experience with timer and navigation
- 3) Enable performance tracking with detailed analytics and reports
- 4) Create multiple test categories for different competitive exams
- 5) Implement secure user authentication and session management
- 6) Design an intuitive and responsive user interface

Secondary Objectives

- 1) Ensure reliable data storage using MongoDB database
- 2) Implement security measures (authentication, authorization using JWT)
- 3) Optimize application for performance and scalability
- 4) Provide error handling and smooth user experience
- 5) Enable content expansion like blog and exam updates

1.4 Scope

In Scope:

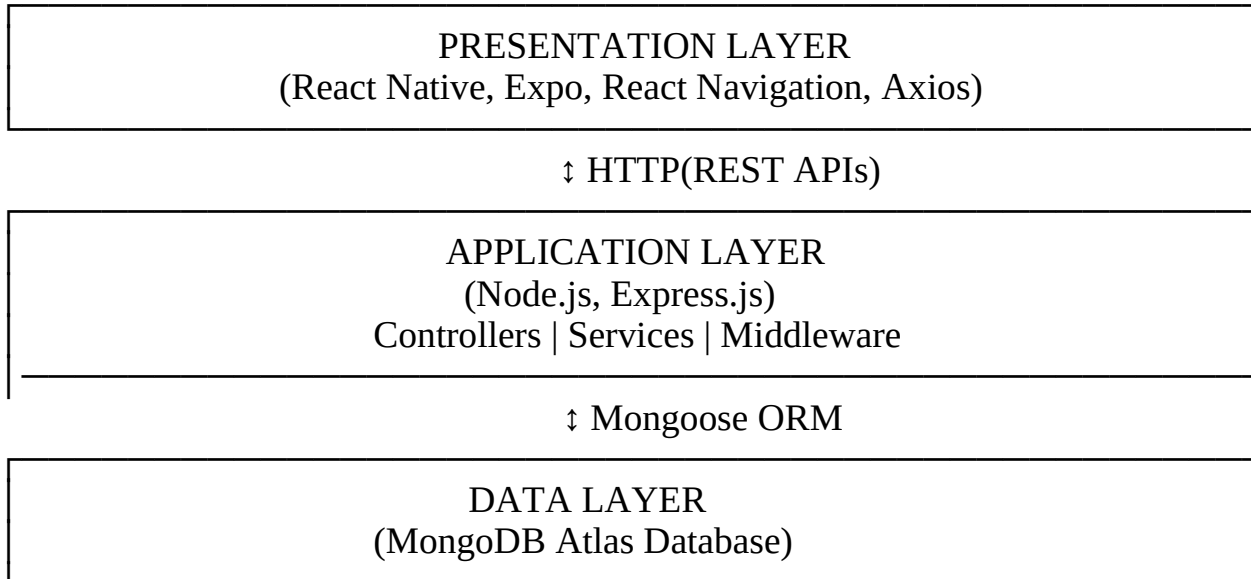
- User registration and authentication
- Secure login/logout functionality using JWT
- Multiple exam categories (SSC, Banking, UPSC, Engineering, etc.)
- Mock test creation and management (Full-length, Sectional, PYQs)
- Real-time test interface with timer and navigation
- Question rendering with multiple choice options
- Test submission and automatic evaluation
- Performance analytics (score, accuracy, progress tracking)
- Personalized dashboard with insights and improvement tips
- Notifications and alert systems.
- Database Storage for Users, tests, exams, and results.
- Responsive web interface

Chapter-2

SYSTEM ARCHITECTURE

2.1 Architecture Overview

The application follows a three-tier architecture:



The system is designed using a three-tier architecture, which separates concerns into three main layers — Presentation Layer, Application Layer, and Data Layer. This structure ensures modularity, scalability, maintainability, and efficient communication between components.

1. Presentation Layer

Technologies Used:

React Native: Builds cross-platform mobile and web UI

Expo: Simplifies development and deployment

React Navigation: Handles routing and screen transitions

Axios: Handles API requests

Description:

The presentation layer serves as the user interface (UI) of the application. It is also responsible for displaying information such as exams, tests, results, and dashboards. And It captures user inputs like test attempts and communicates with the backend using REST APIs.

2. Application Layer

Technologies Used:

Node.js: JavaScript runtime for backend logic

Express.js: Framework for building REST APIs

JWT (JSON Web Token): Handles authentication and authorization

Mongoose: ODM for MongoDB

Components:

Controllers: Handle incoming API requests and responses

Middleware: Handles authentication and validation

Services: Contain the core business logic like Test Evaluation.

Models: Define database schemas

Description:

The Application Layer acts as the core of the system — processing data, executing business rules, and managing communication between the presentation and data layers.

It ensures that all application operations such as user authentication, transaction handling, and data synchronization are carried out efficiently.

3. Data Layer

Technologies Used:

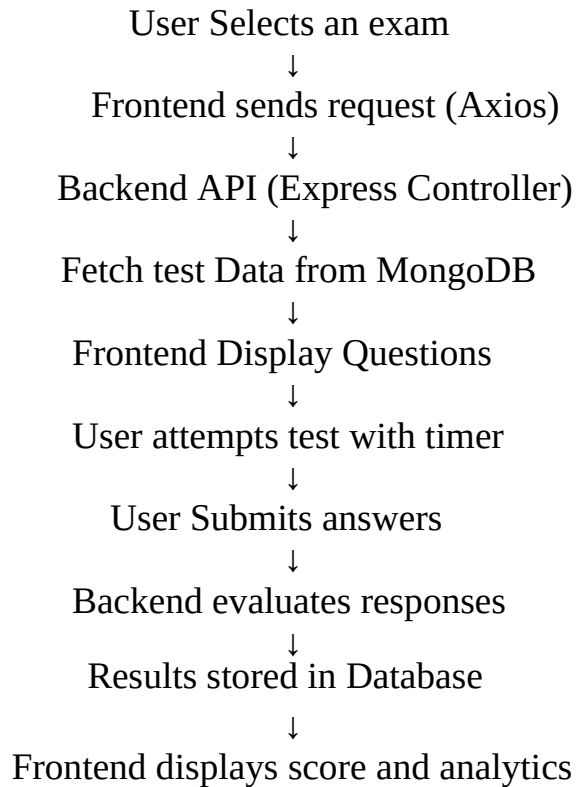
MongoDB Atlas: Cloud database for storing application data

Mongoose: Provides schema-based modeling

Description:

The Data Layer is responsible for data storage, retrieval, and persistence. It abstracts raw database interactions and provides clean interfaces for the application layer to work with data.

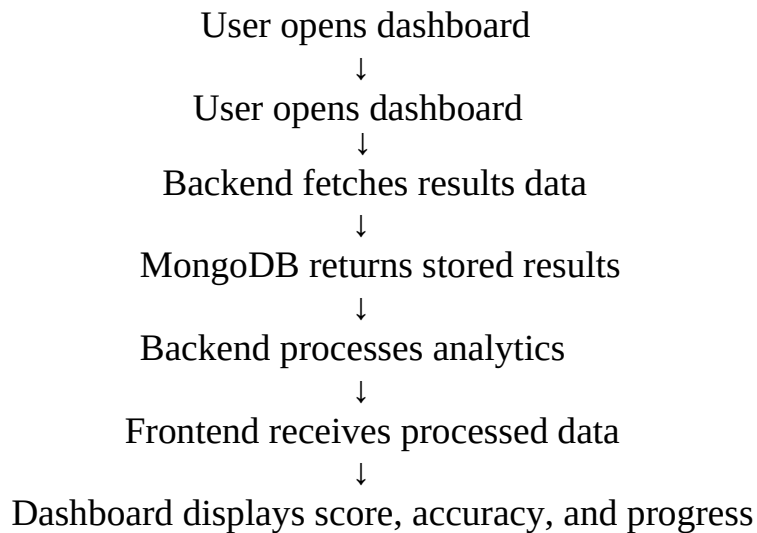
2.2 Mock Test Flow



Real-Time Mock Test Flow

When a user submits a mock test, the frontend sends the responses to the backend. The backend processes the answers, calculates score, accuracy, and performance metrics, and stores the results in MongoDB. The evaluated data is then sent back to the frontend, where it is displayed on the dashboard in real-time.

Dashboard & Analytics Flow



Real Time Dashboard & Analytics Workflow

When the user navigates to the dashboard, the frontend sends a request to the backend for performance data. The backend retrieves the user's test results from MongoDB and processes key metrics such as average score, accuracy, and progress trends. This processed data is then sent to the frontend, where it is visually displayed through charts and summaries to help the user analyze their performance.

2.3 Design Patterns Used

MVC (Model-View-Controller) – Separates UI, logic, and data

REST Architecture – Standard API communication

Middleware Pattern – Handles authentication and validation

Repository Pattern – Data access abstraction using Mongoose

Singleton Pattern – Database connection management

Chapter-3

TECHNOLOGY STACK

Frontend Technologies:

React Native, Expo, JavaScript, React Navigation, Axios

Backend Technologies:

Node.js, Express.js, Mongoose, JWT (JSON Web Token)

Database:

Mongos Atlas

Build & Tools:

Node Package Manager (NPM), Git, GitHub, Visual Studio Code

Communication Protocols:

HTTP (REST APIs)

Security & Utilities:

JWT Authentication, bcrypt.js, dotenv

FEATURES AND FUNCTIONALITY

4.1 User Management

4.1.1 User Registration

Input Fields: Username, Email, Password

Validations:

Username: Unique, 3-50 characters

Email: Valid format, unique

Password: Minimum 6 characters (encrypted with BCrypt)

Process:

User submits registration form

Backend validates uniqueness

Password encrypted using BCrypt

User saved to database

JWT token generated for authentication

Success message displayed

Redirect to login page

4.1.2 User Authentication

Login Process:

User enters username and password

Backend validates credentials

Password verified using BCrypt

JWT token generated and sent to frontend

User session stored (token in local storage)

Redirect to home/dashboard

Logout Process:

User clicks logout button

JWT token removed from storage

Redirect to login page

Session cleared

4.2 Exam & Test Management System

4.2.1 Exam Search

Functionality:

Search by name or category

Real-time filtering of exams

Displays available exam categories (SSC, Banking, UPSC, etc.)

Shows popular and featured exams.

Allow quick navigation to test series.

4.2.2 Test Selection Workflow

Step 1: Select Exam

- User selects an exam category
- Clicks on desired exam
- System displays available mock tests

Step 2 View Test

- Displays list of tests (Full-length, Sectional, Previous Year)
- Shows test details (duration, questions, attempts)

Step 3: Start Test

- User selects a test.
- User clicks “start test”.
- Test interface loads with timer.

Step 4: Submit Test

- User Completes test.
- Submit answers
- Backend evaluates responses

Step 5: View Results

- Score and accuracy displayed
- Performance stored in database
- Users can analyse results

4.2.3 Test

Display:

List of available mock tests

Test type and duration

Number of questions and marks

Features:

Sort tests by difficulty or popularity

Filter tests by type

Free and premium test indicators

4.3 Test Interface & Question System

4.3.1 Test types

A. Full Length Tests

Purpose: Simulate real exam environment

Access: Available for all users

Features:

Covers all sections of the exam

Time tested environment

Real exam pattern simulation

Auto Submission on time completion

B. Sectional Test

Purpose: Practice Specific Sections

Access: Available for all users

Features:

Focus on individual subjects (Quant, Reasoning, English, etc.)

Shorter duration

Section-wise performance Analysis

C. Previous Year Tests

Purpose: Practice actual exam questions

Access: Available for all users

Features:

Based on previous exam patterns

Helps Understands question trends

Real exam difficulty level

4.3.2 Question Types

1. Multiple Choice Question (MCQ)

Text-based questions with 4 options

Single correct answer selection

Instant option selection and navigation

Question numbering and indexing

2. Test features

Timer display for entire test

Question Palette for Navigation

Mark for review Option

Submit test option with confirmation

3. Answer Handling

Features:

User can select one option per question
Answers are saved automatically
User can modify answers before submission
Marked answers are highlighted
Supports marks for functionality

4. Navigation System

Features:

Next and previous navigation
Direct question selection via Palette
Answered / Unanswered indicators
Smooth navigation between questions

5. Timer & Submission

Features:

Timer displayed throughout the test
Auto submission when time ends
Manual submission options
Confirmation before final submission

4.4 Real Time Features

4.4.1 Test Evaluation

Technology: Backend evaluation using Node.js & Express.js

Connection:

Triggered on test submission
Backend processed answers correctly
Score calculated in real time and Results stored in Database

4.4.2 Performance Tracking

Score Tracking:

Calculated after each test
Maintained in User Dashboard
Display accuracy and performance trends

Progress Tracking:

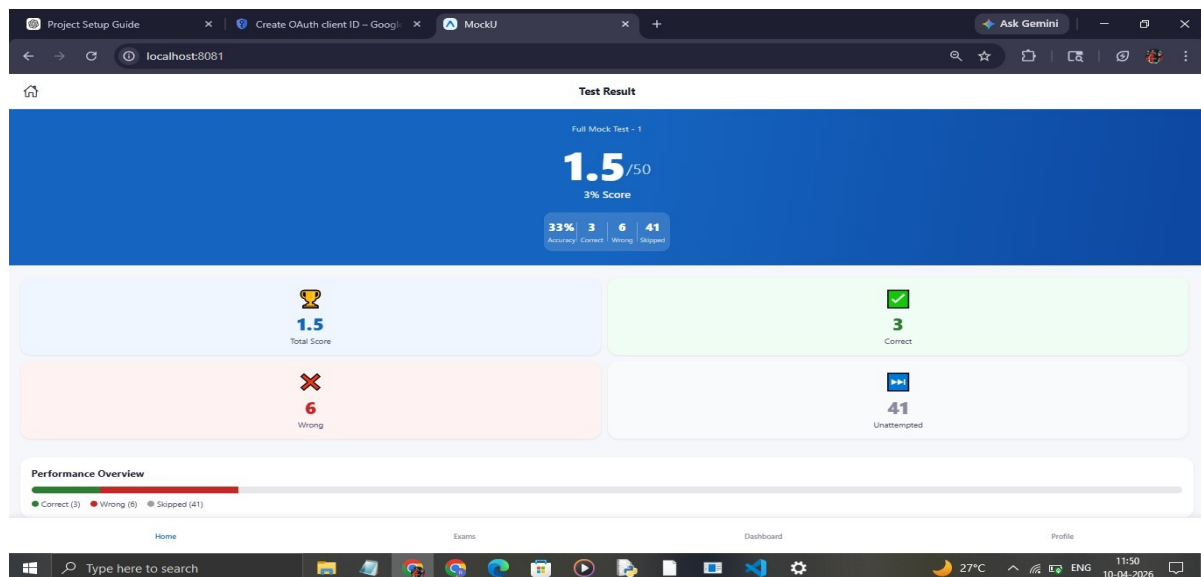
Track number of tests taken
Updates Average score dynamically
Identifies strong and weak areas

4.4.3 Test Submission & Result Delivery

Process:

User completes a mock test and clicks submit
Frontend sends response to backend API
Backend evaluates Answers
Score and Accuracy calculated
Results are stored in database
Backend sends evaluated response to frontend
Frontend displays result instantly without refresh

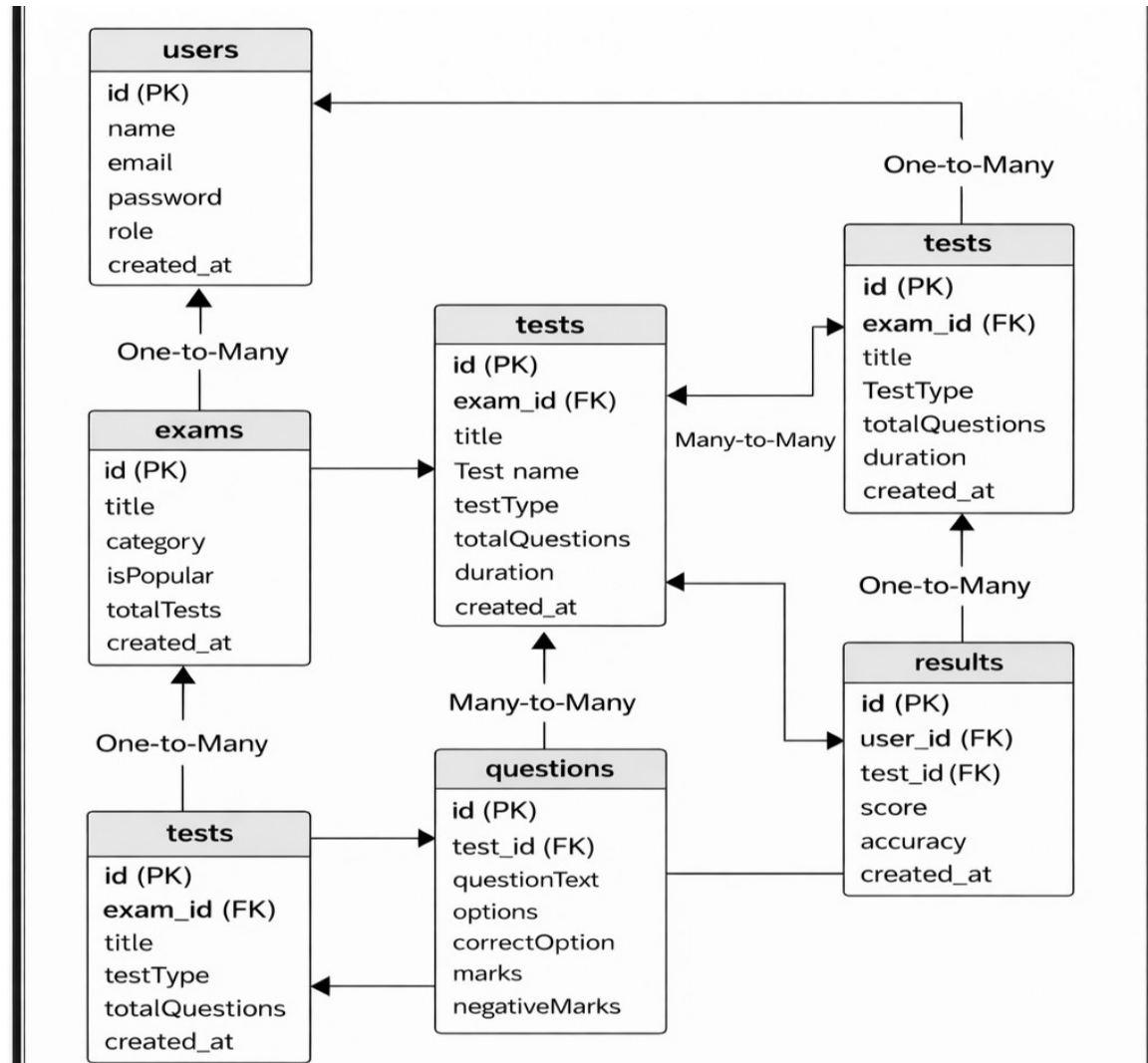
Delivery Time: < 1s typically



Chapter-5

DATABASE DESIGN

5.1 Entity Relationship Diagram (ERD)



5.2 Table Schemas

5.2.1 users

Columns:

id - Primary key - (auto generated object)
name – User full name
password - BCrypt hashed password
email - User's email address
Role – User role (User / Admin)
created_at - Registration timestamp

5.2.2 Exams

Columns:

id - Primary key
title – Full exam name
Category – Exam Category
ispopular – Boolean flags for trending exams
Is featured – Boolean flag for homepage highlight
TotalTests – Number of mock tests available
TotalAttempts – Total attempts taken
created_at - Timestamp

5.2.3 Tests

exam_id – Foreign key – exams.id
Id – Primary key
title – Test title
TotalQuestions – total number of questions
Duration – Duration in minutes

5.2.4 Questions

Id – Primary key
test_id – Foreign key – test.id
QuestionText – Question Statement
Options – Array of options (A,B,C,D)
CorrectOption – Correct answer key
Explanation – explanation for answer
Section – Subject (Quant, reasoning)
Difficulty – easy / medium / hard
Marks – Marks for correct answer
Negative_marks – Negative marking value

5.2.5 Results

Id – Primary key
user_id – Foreign key – user.id
test_id – Foreign key – test.id
Score – Marks obtained
Accuracy – Percentage Accuracy
TimeTaken – time taken on test
created_at - Timestamp

5.3 Database Relationships

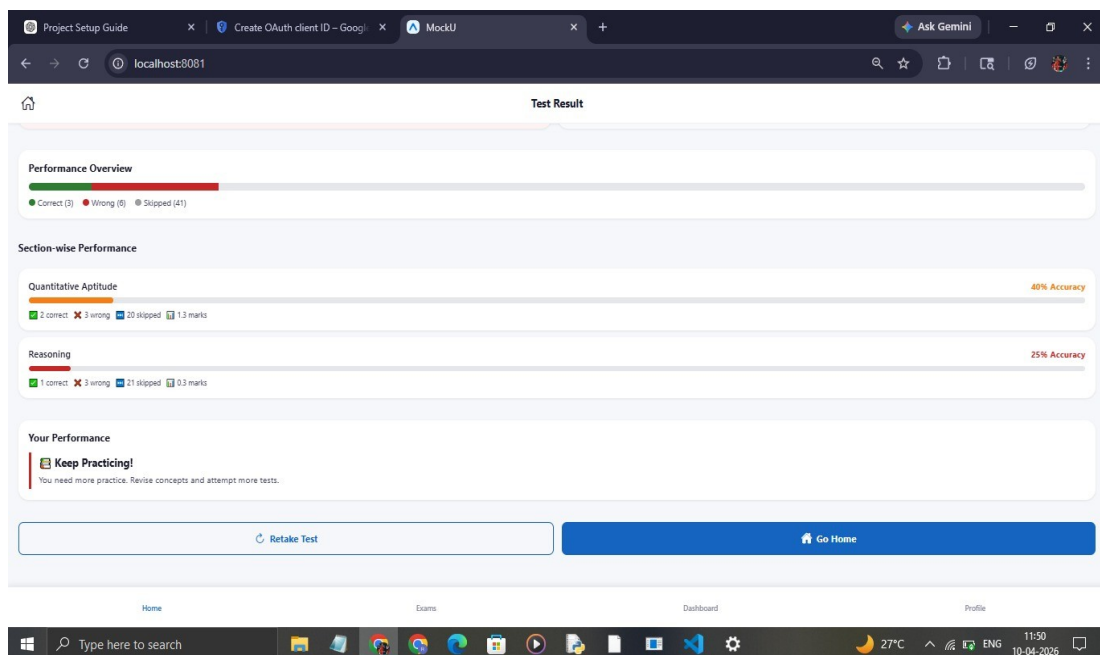
Relationship	Type	Description
users → results	One-to-Many	User attempts multiple tests
exams → tests	One-to-Many	One exam contains multiple tests
tests → questions	One-to-Many	One test contains many questions
users → results → tests	Many-to-Many	Users attempt multiple tests
tests → results	One-to-Many	Test has multiple user attempts

The database design defines several relationships among the key entities to maintain data integrity and ensure smooth interaction between users, exams, tests, and question.

The users table has a one-to-many relationship with the mock test results table, indicating that a single user can attempt multiple tests over time and each test is kept as a Separate entity. Similarly Users also have a one-to-many relationship with test through result Which insists each test can be attempted by multiple users.

The relationship between exams and tests is many-to-many, where each exam category can have multiple mock tests associated with it. The tests to questions relationship is one-to-many, meaning each test contains many multiple questions covering different sections like Quantitative Aptitude, Reasoning, etc.

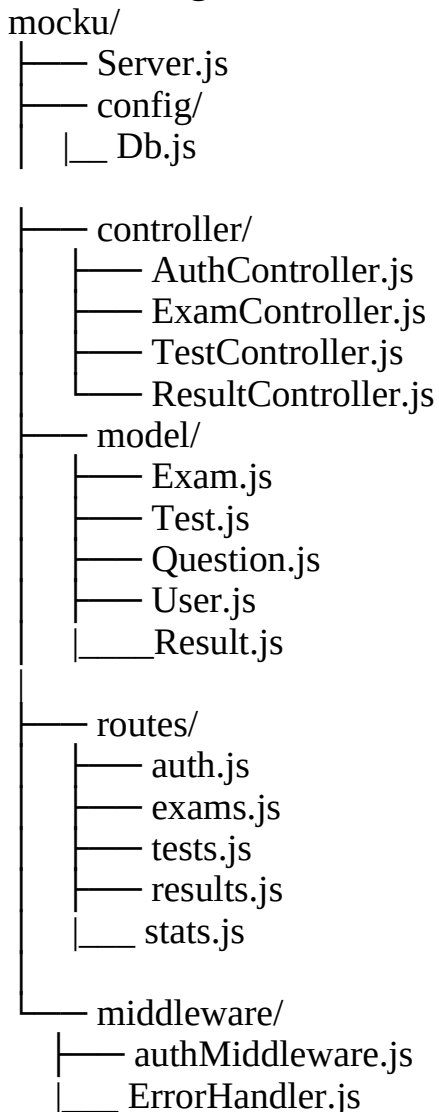
Additionally, each test maintains one-to-many relationship with results, as multiple Users can attempt the same test and generate different scores and performance data. Overall, these relationships together form a robust and scalable data structure that supports mock test creation, result generation and performance rating functionality within the application.



IMPLEMENTATION DETAILS

6.1 Backend Architecture

6.1.1 Package Structure



6.1.2 Key Components

Server.js

- Initializes Express application
- Connects to MongoDB using Mongoose
- Applies middleware (CORS, Helmet, Morgan)
- Registers API routes
- Handles global errors

authController.js

- Handles user registration and login
- Uses bcrypt for password hashing
- Generates JWT tokens for secure authentication
- Protects routes using middleware

Seeder.js

- Populates database with:
 - Users (admin + testUser)
 - Tests and Questions
- Helps in quick testing and deployment

API routes

- /api/auth → Login & Register
- /api/exams → Fetch exam categories
- /api/tests → Manage mock tests
- /api/results → Store user performance

Database Models (mongoose schema)

- Defines structure for Users, Exams, Tests, Questions, Results
- Handles relationships using ObjectId references
- Enables validation and schema enforcement

6.2 Frontend Architecture

6.2.1 File Structure

```
src/  
├── App.js  
├── Navigation.js  
└── api.js
```

6.2.2 Key JavaScript Functions

Connection Management:

axios.create() – Creates API instance
setAuthToken() – Attaches JWT token to request
handleError() – Handles API errors

Navigation Options:

navigateToScreen() – Switch between screens
TabNavigator – Bottom tabs (Home, Exams, Dashboard, Profile)
StackNavigator – Handles inner navigation

User Operations:

loginUser() – Handles login
registerUser() – Handles signup
logoutUser() – Clears session

Test Operations:

fetchExams() – Loads exam categories
fetchTests() – Loads tests
startTest() – Starts test
submitTest() – Submits answers

Dashboard Operations:

refreshDashboard() – Refresh UI

Chapter-7

USER INTERFACE DESIGN

7.1 Design Principles

Simplicity - Clean, uncluttered interface

Responsiveness - Works on all screen sizes

Intuitiveness - Easy to navigate and use

Feedback - Visual feedback for all actions (test submit, login, etc.)

Consistency - Uniform design across pages

Accessibility - Readable fonts, good contrast

7.2 Color Scheme

Primary Color: #1E5BB8 (Deep Blue – main branding)

Secondary Color: #2563EB (Bright Blue – gradient end)

Success Color: #22C55E (Green – correct answers, success states)

Error Color: #EF4444 (Red – wrong answers, errors)

Warning Color: #F59E0B (Orange – alerts, tips)

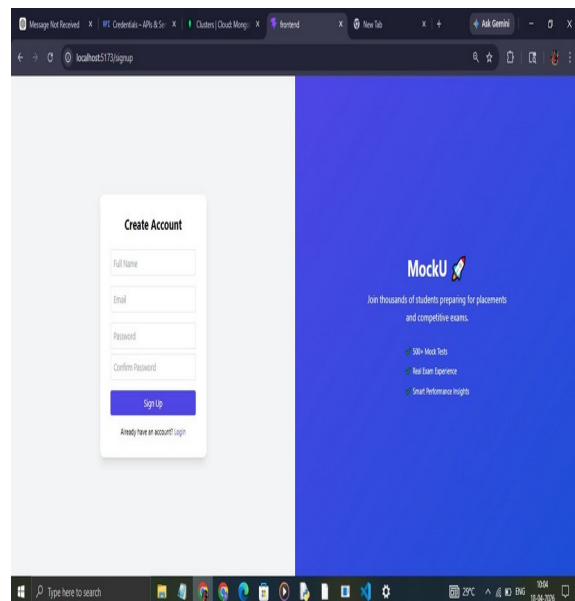
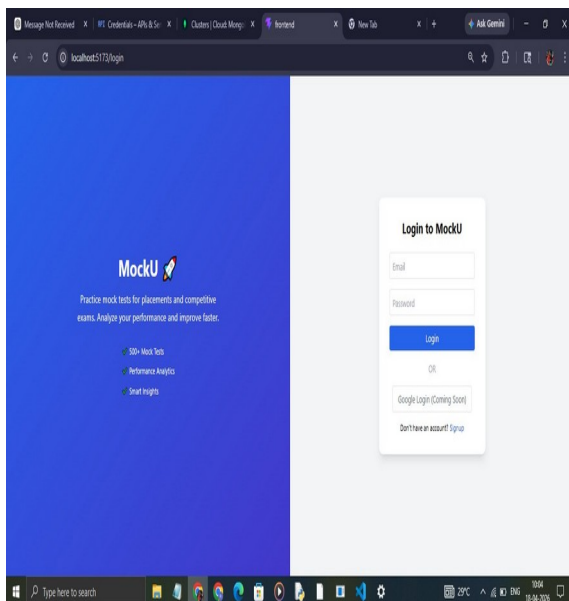
Background: #F5F7FB (Light Gray – main background)

Text: #1F2937 (Dark Gray – primary text)

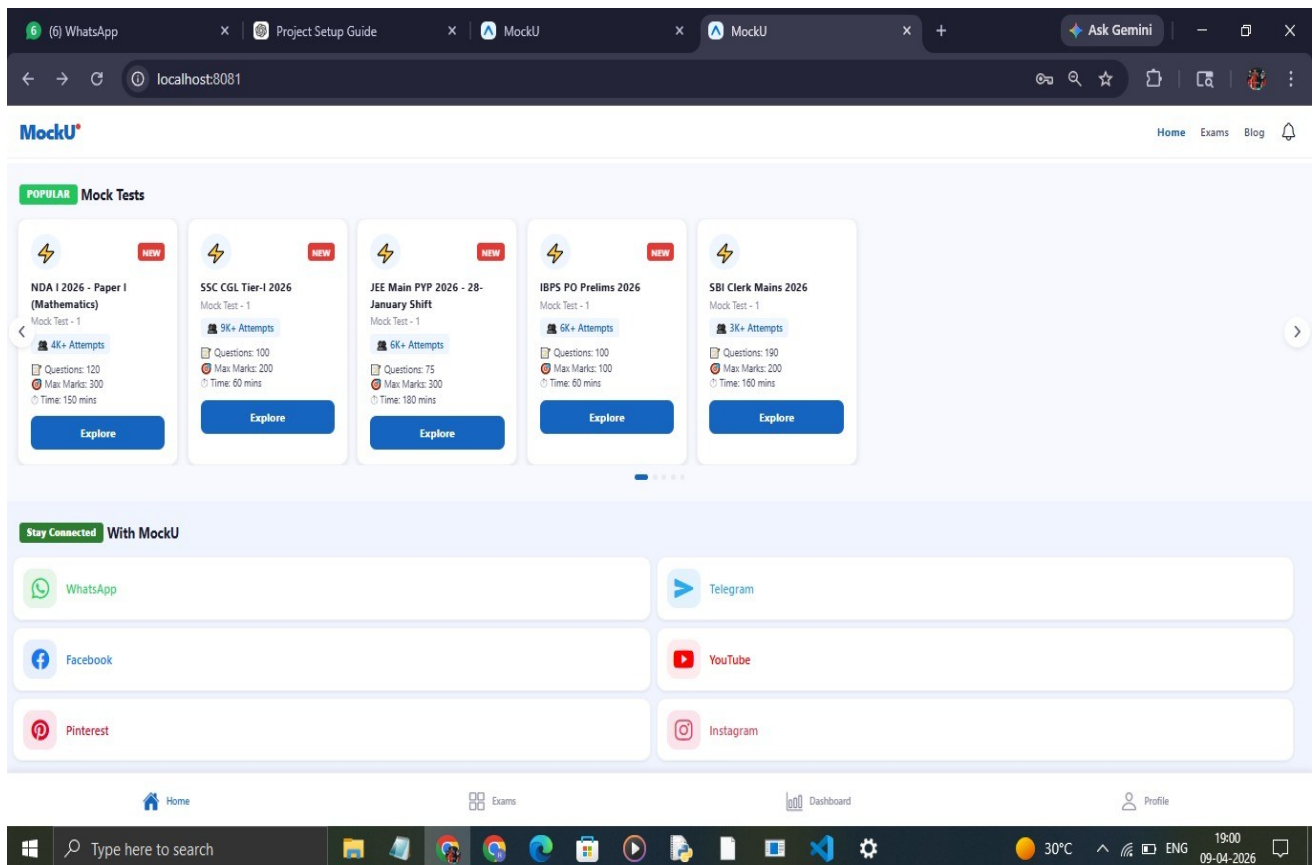
Border: #E5E7EB (Light Gray – borders, dividers)

7.3 Layout Structure

Login/Register Pages:



Main Mock Test Interface:



7.4 Responsive Design

Desktop (>992px):

Two-column layout (sidebar with exam categories + main content area)
Full features visible (mock tests, performance stats, navigation)
Dashboard charts and analytics displayed clearly

Tablet (768px - 992px):

Collapsible sidebar for exam categories
Full-width content when sidebar is hidden
Touch-friendly buttons and cards

Mobile (<768px):

Bottom navigation bar (Home, Exams, Dashboard, Profile)
Single column layout for better readability
Optimized test interface for small screens
Simplified UI for faster performance

SECURITY FEATURES

8.1 Authentication & Authorization

8.1.1 Password Security

Security Level:

Passwords are encrypted using BCrypt hashing (10 salt rounds)

Salted hashing (prevents rainbow table attacks)

Unique salt per password

Session Security:

JWT token-based authentication

Token stored securely (localStorage/sessionStorage)

Automatic logout on token expiration

Server-side verification on every protected API call

8.2 Data Validation

8.2.1 Input Validation (Frontend)

Input validation is performed on the frontend to ensure that user-provided data is correct before being sent to the backend. This improves user experience and reduces unnecessary server requests.

In MockU, validation is applied in:

 Login/Register Forms:

 Email must follow valid format

 Password must meet minimum length requirements

 Required fields cannot be empty

 Test Interface:

 Ensures valid option selection

 Prevents submission of incomplete responses

 Profile Updates:

 Validates user inputs before updating data

Validation messages are displayed instantly, helping users correct errors efficiently.

8.2.2 Backend Validation (Server-Side)

Backend validation ensures that all incoming data is verified before processing or storing in the database. This prevents invalid or malicious data from entering the system.

In MockU:

 Validation is implemented using Mongoose schema validation

 Required fields, data types, and formats are strictly enforced

 APIs validate request payloads before execution

FUTURE ENHANCEMENTS

Out of Scope:

Adaptive Testing (AI-Based):

The current system provides standard mock tests. Adaptive testing, where question difficulty adjusts based on user performance, is not implemented in this version.

Real-Time Collaboration / Discussion:

Users cannot currently discuss questions or solutions with others in real time. A peer discussion or doubt-solving feature is not included.

Advanced Performance Analytics:

Basic performance metrics (score, accuracy, etc.) are available. Advanced analytics such as AI-based insights, rank prediction, and personalized study plans are not implemented.

Offline Mode:

The application requires an active internet connection. Offline test-taking and data sync features are not supported in this version.

Mobile Native Application:

The system is currently a web-based application. Dedicated Android/iOS apps are not developed yet.

Push Notifications:

Real-time notifications for new tests, results, or reminders are not integrated in the current version. All mock tests are currently free. Paid test series, subscriptions, and payment gateway integration are not implemented.

Multi-Language Support:

The platform currently supports limited language options. Full multi-language support for tests and UI is not available.

Bookmark & Revision System:

Users cannot bookmark questions or create custom revision sets for later practice

CONCLUSION

The MockU platform successfully demonstrates the development of a comprehensive online mock test system designed to help students prepare for competitive exams. It provides essential features such as user authentication, exam categorization, mock test interfaces, performance analysis, and result tracking — ensuring an effective and structured and with preparation experience.

The application follows a well-defined three-tier architecture consisting of a good presentation layer (frontend), application layer (backend), and data layer (database). This is an architecture ensures scalability, maintainability, & efficient data handling. Technologies such as React/Expo (frontend), Node.js with Express (backend), and MongoDB (database) are effectively integrated to deliver a seamless user experience.

Throughout the development process, key aspects such as data validation, secure authentication using JWT, and efficient database management were implemented to ensure reliability and consistency. Although advanced features like AI-based recommendations, adaptive testing, and mobile-native applications are currently out of scope, the system will establish a strong foundation for future enhancements.

Overall, the project achieves its objective of building a functional and user-friendly mock test platform. It reflects a practical understanding of full-stack development, REST - API integration, and database design — making it a scalable solution for real-world useful educational applications.

REFERENCES

1. Mozirilla Developer Network (MDN). (2025).
Web Development Documentation (HTML, CSS, JAVASCRIPT).
Retrieved from <https://developer.mozilla.org/>
2. Oliveboard. (2025).
Oliveboard Mock Test Platform
Retrieved from <https://www.oliveboard.in>
3. Mockers. (2025).
Mockers – All in one Mock Tests
Retrieved from <https://www.mockers.in>
4. MongoDB INC. (2025).
MongoDB Atlas Documentation
Retrieved from <https://www.mongodb.com/docs/atlas/>
5. Node.js Foundation. (2025).
Node.js Documentation
Retrieved from <https://nodejs.org/en/docs>
6. React.js Team. (2025).
React Documentation
Retrieved from <https://react.dev/>
7. Express.js (2025).
Express web Framework Documentation
Retrieved from <https://expressjs.com/>